Multicorner Timing Analysis Acceleration for Iterative Physical Design of ICs

Wei W. Xing, *Member, IEEE*, Longze Wang, *Student Member, IEEE*, Zhelong Wang, *Student Member, IEEE*, Zhaoyu Shi, *Student Member, IEEE*, Ning Xu, *Member, IEEE*, Yuanqing Cheng, *Senior Member, IEEE*, and Weisheng Zhao, *Fellow, IEEE*

***Abstract*—We propose a multicorner multistage timing analysis prediction framework using a generalized linear model with latent features. We then further improve such methods using kernel trick extension, transfer learning with knowledge from previous designs, and multioutput feature engineering to deliver state-of-the-art (SOTA) prediction accuracy with very limited training data. Most importantly, our method is equipped with a Bayesian decision strategy to deliver reliable predictions with accuracy close to 100%, pushing the frontier of the machine-learning-based static timing analysis (STA) for practical implementation in the industry environment, where reliability is highly desired. Experimental results show that the accuracy of our proposed method outperforms the SOTA competitors by up to 4x and can improve prediction accuracy to 100% with little extra STA executions.**

***Index Terms*—Active learning, Bayesian decision theory, featured engineering, Gaussian process, machine learning, multiprocess corners, static timing analysis (STA).**

I. Introduction

IN ORDER to guarantee the timing yield, chip designers resort to static timing analysis (STA) to evaluate the timing correctness under different process corners (PCs) [1]. Thanks to the fast speed and reliability of STA, it has been widely used for timing verification in the past few decades. As the technology node scales down into the nanometer regime, the process, voltage, and temperature (PVT) variations impact chip performance significantly [2]. As a result, the number of process corners increases drastically with the technology node scaling [3], [4]. For instance, the number of process corners rockets up to over 100 in the 20-nm technology node and below [5]. The explosion of process corner count presents a challenge for traditional STA—extremely high-computational cost. Even worse, the conventional physical design goes through synthesis, floorplan, power plan, placement, clock synthesis, routing, and engineering change order (ECO) optimizations. Each stage requires extensive timing simulations to pinpoint the timing failure as early as possible. Therefore, STA has become an ever-increasing time-consuming procedure in modern chip timing verification.

To reduce the timing analysis overhead, several effective techniques have been proposed, which can be mainly classified into two categories. One is to reduce the time to that of one process corner analysis [6], [7]. More specifically, Silva et al. [6] used a branch-and-bound method to obtain the worst-case process corner for chip sign-off timing analysis. Based on a similar idea, Onaissi and Najm [7] derived the worst-case timing results much faster than a normal multicorner STA. By estimating the worst-case timing among PCs, the upper bound of the worst timing of the critical path can be evaluated with one-pass analysis instead of running STA for all process corners. However, the upper bound may not be a strictly tight bound, leading to an overdesign with a conservatively large design margin. Moreover, the worst case may not occur in practical operational conditions because the combinations of PVT leading to the worst-case delay are less likely to occur simultaneously. Another category of accelerating STA is taking the process and environment parameters as independent random variables, and using statistical techniques for one-pass timing analysis, such as [8], [9].

Recently, inspired by the success of machine learning in EDA fields, researchers and industry have started to investigate the use of data-driven techniques to accelerate STA. Andrew et al. [10] proposed a multivariate linear regression technique to predict path timings of "unobserved corners" with path timings of "observed corners." The intuition is that the timing result of all corners lives in a low-dimensional ambient space and thus can be reconstructed using timing results under some corners by harnessing the correlation under all corners. Following the same assumption, Zhao et al. [11] put forth nonlinear regression models for the STA prediction with improved accuracy. Also, they proposed using a statistical analysis tool to pick key STA results (referred to as dominating process corners) as model inputs to save extra STA executions while maintaining good predictive accuracy.

Manuscript received 16 July 2023; revised 20 October 2023 and 20 December 2023; accepted 17 January 2024. Date of publication 2 February 2024; date of current version 20 June 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 92373205, and in part by the Natural Science Foundation of Beijing, China, under Grant Z230002. This article was recommended by Associate Editor L. Behjat. *(Wei W. Xing and Longze Wang contributed equally to this work.) (Corresponding author: Yuanqing Cheng.)*

Wei W. Xing, Longze Wang, Zhaoyu Shi, Yuanqing Cheng, and Weisheng Zhao are with the School of Integrated Circuit Science and Engineering, Beihang University, Beijing 100191, China, and also with the National Key Laboratory of Spintronics, International Innovation Institute, Beihang University, Hangzhou 311115, China (e-mail: wxing@buaa.edu.cn; wanglongze@buaa.edu.cn; shizhaoyu@buaa.edu.cn; yuanging@buaa.edu.cn; weisheng.zhao@buaa.edu.cn).

Zhelong Wang and Ning Xu are with the School of Information Engineering, Wuhan University of Technology, Wuhan 430070, China (e-mail: wangzhelong@whut.edu.cn; xuning@whut.edu.cn).

Digital Object Identifier 10.1109/TCAD.2024.3361401

1937-4151 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

Despite its success, this type of method is limited because 1) it requires many data to correctly choose its dominant process corners; 2) it assumes a simple model for all circuits, which is neither realistic nor accurate; 3) the model can not update itself when it is used in an iterative optimization process and may eventually lead to poor accuracy; 4) the dominant process corners are prefixed and cannot be adapted with more information coming into the system; 5) the model lacks the ability to tell the confidence of the prediction accuracy, which makes it unrealistic to be deployed in a practical chip design where robustness is crucial.

To resolve these issues, we solve the STA prediction problem around all main physical design stages in a unique space taking advantage of their close correlations. We propose a multilinear regression model with implicit latent features to harness the historical information and predict the STA results accurately. We then generalize our model as a nonlinear model to deal with complex STA problems along with a scalable and efficient model training procedure. Besides providing more accurate prediction than traditional methods, the proposed method provides a Bayesian decision strategy based on the predictive distribution to actively update the model for uncertain predictions and improve the prediction accuracy to nearly 100% with little cost. This way, the timing predictions are given with confidence, and the results are reliable, making the proposed method more practical to be implemented in a real industrial design. Furthermore, feature engineering is added to our method to further improve the prediction accuracy.

In summary, our contributions are listed as follows.

1) We propose a novel flexible framework for the STA prediction problem with latent features that enable knowledge transfer from previous experimental data.
2) We establish a Bayesian nonlinear model that is accurate, scalable, and flexible for timing prediction.
3) We put forth multioutput feature engineering (MoFe) to further improve model accuracy.
4) We proposed a Bayesian decision strategy to assist STA, enhancing the reliability of STA predictions with active correction such that the model accuracy is almost 100%, making the proposed method, more practical for real chip designs.

The remainder of this article is organized as follows. Section II formulates the STA prediction problem and analyzes the drawbacks of the classic approach, which motivates this work. Section III describes the proposed method with detailed key techniques involved. Section IV illustrates the experiment results and Section V concludes our work.

II. Problem Formulation and Preliminaries

A. Problem Formulation

Timing closure is a critical requirement in the chip design process, which is to ensure that the timing constraints are met for all paths in the design. The path delays in a given integrated circuit are mainly influenced by process (P), voltage (V), and temperature (T), which are known as PVT variations. In multicorner STA, the STA results under PCs, which are key combinations of PVT variations, are estimated and compared with the timing constraints to ensure the timing closure [12]. The particular combinations are generally encoded in a design library [13] with ID numbers from 1 to I. We denote a particular circuit design using vector $\mathbf{z}$ (of which the value is unknown), which contains J critical paths. The timing delay of the jth path under the ith PC for the circuit $\mathbf{z}$ is denoted as $y(\mathbf{z}, i, j)$. In a normal STA, we need to run STA to get the full STA matrix $\mathbf{Y}(\mathbf{z}) \in \mathbb{R}^{I\times J}$ to ensure the timing closure. The machine learning-based STA prediction problem is to predict the STA matrix $\mathbf{Y}(\mathbf{z})$, with some sampling data $\mathcal{D} = \{\mathbf{Y}(\mathbf{z}_1), \mathbf{Y}(\mathbf{z}_2), \ldots, \mathbf{Y}(\mathbf{z}_N)\}$.

B. Classic STA Prediction Methods

Ideally, we would like to directly predict the STA matrix $\mathbf{Y}(\mathbf{z})$ using a machine learning model for any given circuit $\mathbf{z}$. However, this is almost an impossible task because we cannot quantify $\mathbf{z}$, i, nor j as model inputs, not to mention their complex interactions. To resolve this challenge, [10], [11] compromise the ultimate goal of directly predicting $\mathbf{Y}(\mathbf{z})$. Instead, they make it a matrix infill task, where the STA matrix $\mathbf{Y}(\mathbf{z})$ is partially observed, and the goal is to predict the missing entries. Denote the ith column of $\mathbf{Y}(\mathbf{z})$ as $\mathbf{y}(\mathbf{z}, i)$. Their method first decides a set of I' ($I' << I$) PCs that matter most in the predictions and uses this as model input for a regression model. The inputs are then $\mathbf{x} = (\mathbf{y}(\mathbf{z}, 1, j), \ldots, \mathbf{y}(\mathbf{z}, I', j))^\top$ whereas the model target output are defined as $\mathbf{t} = (\mathbf{y}(\mathbf{z}, I' + 1, j), \ldots, \mathbf{y}(\mathbf{z}, I, j))^\top$. This way, each row of $\mathbf{Y}(\mathbf{z})$ in sampling data $\mathcal{D}$ produces a model input and corresponding output pair to train a regression model. Note that the selection of the corner can be arbitrary without following an ordinal order. The goal is to approximate the mapping from the known STA results $\mathbf{x}$ to the unknown results t with existing data

$$\mathbf{t} = \mathbf{g}(\mathbf{x}) \tag{1}$$

where $\mathbf{g}$ represents the mapping, which can be a linear or nonlinear regression model. Due to the fact that $\mathbf{x}$ and $\mathbf{t}$ are highly correlated, Zhao et al. [11] showed the efficiency of such an approach in successfully predicting y, which reduces the total number of STA simulations by a ratio of $1 - (I'/I)$.

Despite its novelty, this method has several limitations. First, the model depends on preselected PCs, which do not generalize well to different circuits. Because the model is trained with a fixed set of PCs, it cannot adapt to different circuits by using other PCs as inputs.

Second, the predictions lack uncertainty quantification. When it makes predictions, it is hard to tell how reliable the predictions are. This hinders it from being used in a real STA flow where wrong predictions can lead to a catastrophe.

III. Proposed Methodology

The big picture of our proposed STA method are illustrated in Fig. 1. As shown in the figure, we first introduce a multilinear regression model with latent features to capture the correlations among timing results of PCs (see Section III-A). Then we improve the model by generalizing it to a nonlinear model (see Section III-B). After that, we present an efficient model training strategy (see Section III-C). Next, we

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

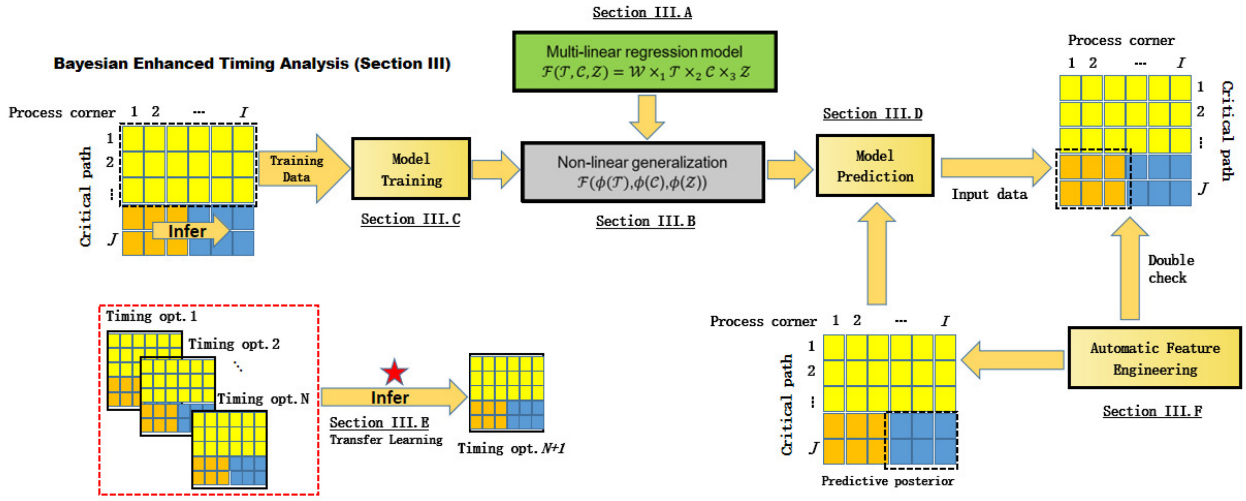

Fig. 1. Workflow of our timing prediction method, including multilinear regression, nonlinear generalization, model training, model prediction, and prediction correction with Bayesian decision theory.

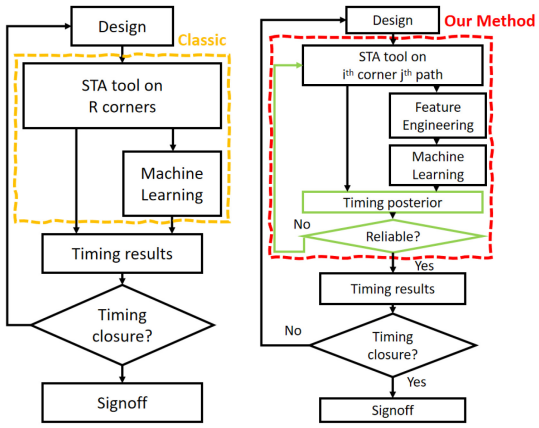

Fig. 2. Main differences between (b) our proposed model and (a) conventional method.

discuss the reasoning process of the model (see Section III-D) and evaluate the prediction confidence for iterative timing optimization results (see Section III-E). At last, we augment our method with multiple output feature engineering (MoFe) (see section Section III-F). The main differences of our model from the classic method are also illustrated in Fig. 2.

A. Linear Regression Model With Latent Features

We first put forth a multilinear regression model, which is flexible and efficient to enhance the existing classic approach for timing prediction. The original goal of predicting $\mathbf{Y}(\mathbf{z})$ is broken down into predicting its components $y(\mathbf{z}, i, j)$ one by one as an equivalent goal. We can first introduce a simple linear regression model

$$y(\mathbf{z}, i, j) = \mathbf{w}^\top [\mathbf{z}, i, j] + b + \epsilon \tag{2}$$

where $\mathbf{w}$ is the weight vector, b is the bias, and $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is the noise term assumed to be Gaussian. Despite its simplicity, this model is inadequate to capture the complex influence of the process corner and critical path to the timing results. In order to capture the complex influence, we introduce latent features for the process corners and critical paths, denoted as $\mathbf{c}_i \in \mathbb{R}^{D_p}$ and $\mathbf{p}_j \in \mathbb{R}^{D_c}$, respectively, where D_p and D_c are the dimension of the latent features as our model hyperparameters. Since we have I process corners and J critical paths, we need a latent feature matrix $\mathbf{P} \in \mathbb{R}^{I \times D_p}$ and $\mathbf{C} \in \mathbb{R}^{J \times D_c}$ to represent the latent features of the process corners and critical paths, respectively. We will discuss how to determine their values later. Right now, we continue to derive the multilinear regression model, which now becomes

$$y(\mathbf{z}, i, j) = \mathbf{w}^\top [\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j] + b + \epsilon. \tag{3}$$

The advantage of this model is mainly the introduction of latent features $\mathbf{c}_i$ and $\mathbf{p}_j$, which, once well characterized, can provide critical yet quantifiable model inputs of the complex influence of the process corner and critical path to the predictions of timing results by learning from historical data.

B. Nonlinear Regression Generalization

Equation (3) is a linear regression model, which is simple and efficient to train. However, it is not flexible enough to capture the complex influence of the process corners and the critical paths to the timing results. To address this issue and deal with practical challenging STA predictions, we extend the linear regression model to a nonlinear regression model.

First, we introduce a feature mapping function $\boldsymbol{\phi}(\cdot)$, which maps the original input features $\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j$ to a higher-dimensional feature space $\boldsymbol{\phi}(\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j)$. If we choose such a mapping appropriately, we can capture the complex influence of the process corners and the critical paths to the timing results. The model becomes

$$y(\mathbf{z}, i, j) = \mathbf{w}^\top \phi(\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j) + \epsilon \tag{4}$$

where the bias term b is absorbed into $\phi(\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j)$ for simplicity. Choosing a proper feature mapping function $\phi(\cdot)$ can be difficult in practice. In this work, we will use the kernel

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

trick [14] to integrate them out. To this end, we first give a general Gaussian prior for the weight vector $\mathbf{w}$ such that $\mathbf{w} \sim \mathcal{N}(0, \mathbf{I})$. We then try to integrate out the weight vector $\mathbf{w}$ by

$$\begin{aligned} y(\mathbf{z}, i, j) &= \int \mathbf{w}^\top \phi\big(\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j\big) \mathcal{N}(\mathbf{w}|0, \mathbf{I})\, d\mathbf{w} + \epsilon \\ &= \mathcal{N}\Big(0, \phi(\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j)^\top \phi(\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j) + \sigma^2\Big). \end{aligned} \tag{5}$$

We can further define a kernel function $k((\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j), (\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j))$ to replace the inner product $\phi(\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j)^\top \phi(\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j)$ in (5), which frees us from the explicit feature mapping function $\phi(\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j)$. The most commonly used kernel function is the Gaussian kernel function, which is defined as

$$k\big(\mathbf{x}, \mathbf{x}'\big) = \theta_0 \exp\Big(\big(\mathbf{x} - \mathbf{x}'\big)^\top \Lambda \big(\mathbf{x} - \mathbf{x}'\big)\Big) \tag{6}$$

where Λ is a diagonal matrix that scales the influence of each dimension in $\mathbf{x}$ and θ_0 is a scaling factor.

This way, we reproduce a Gaussian process regression (GPR [15]) model tailored for our STA prediction problem. Note that the latent features $\mathbf{c}_i$, $\mathbf{p}_j$ and the hyperparameters Λ and θ_0 are to be determined.

C. Efficient Model Training With Tensor Algebra

The key thing with GPR is that any observations of the function $y(\mathbf{z}, i, j)$ form a joint Gaussian distribution. For all observations of data $\mathcal{D}$, we can write the joint distribution as

$$\mathbf{y} = \mathcal{N}\Big(0, \mathbf{K} + \sigma^2 \mathbf{I}\Big) \tag{7}$$

where $\mathbf{y}$ is a vector of all observations of $y(\mathbf{z}_n, i, j)$ for $n = 1, \ldots, N; i = 1, \ldots, I; j = 1, \ldots, J$, and $\mathbf{K}$ is a $NIJ \times NIJ$ covariance matrix based on the kernel function and the corresponding input features $(\mathbf{z}_n, \mathbf{c}_i, \mathbf{p}_j)$. We can write down the joint likelihood

$$\mathcal{L} \propto \mathbf{y}^\top \Big(\mathbf{K} + \sigma^2 \mathbf{I}\Big)^{-1} \mathbf{y} - \log\det\Big(\mathbf{K} + \sigma^2 \mathbf{I}\Big) \tag{8}$$

where $\det(\cdot)$ is the determinant. Unless there is a model we can use to characterize a netlist as $\mathbf{z}$, we also treat $\mathbf{z}_n$ as latent features. We can optimize $\mathcal{L}$ with respect to model parameters Λ, θ_0, and all latent features $\{\mathbf{Z}, \mathbf{C}, \mathbf{P}\}$.

In this work, we are concerned with STA matrix with the same paths. Thus, the number of paths is the same for all $\mathbf{Y}(\mathbf{z})$.

We can accelerate the computation using tensor algebra [16]. Essentially, we use a separable kernel [17]

$$k\Big((\mathbf{z}, \mathbf{c}_i, \mathbf{p}_j), (\mathbf{z}', \mathbf{c}_i', \mathbf{p}_j')\Big) = k_1\big(\mathbf{z}, \mathbf{z}'\big) k_2\big(\mathbf{c}_i, \mathbf{c}_i'\big) k_3\Big(\mathbf{p}_j, \mathbf{p}_j'\Big) \tag{9}$$

where $k_1(\cdot)$, $k_2(\cdot)$, and $k_3(\cdot)$ are the kernel functions for the three latent features $\mathbf{z}$, $\mathbf{c}_i$, and $\mathbf{p}_j$, respectively.

If we arrange $\mathbf{y}$ in the particular following way $\mathbf{y} = (y(z_1, c_1, p_1), \ldots, y(z_1, c_1, p_J), \ldots, y(z_1, c_I, p_J), \ldots, y(z_N, c_I, p_J))$, then we can write the covariance matrix $\mathbf{K}$ as

$$\mathbf{K} = \mathbf{K}_1 \otimes \mathbf{K}_2 \otimes \mathbf{K}_3 \tag{10}$$

where $\otimes$ is the Kronecker product, $\mathbf{K}_1 = k_1(\mathbf{Z}, \mathbf{Z})$ is the covariance matrix for all $\mathbf{Z}$, similarly, $\mathbf{K}_2 = k_2(\mathbf{C}, \mathbf{C})$ (i.e., $[K_2]_{ab} = k_2(\mathbf{c}_a, \mathbf{c}_b)$) and $\mathbf{K}_3 = k_3(\mathbf{P}, \mathbf{P})$ are the covariance matrices for all $\mathbf{C}$ and $\mathbf{P}$.

Most importantly, we have

$$\mathbf{K}^{-1} = \mathbf{K}_1^{-1} \otimes \mathbf{K}_2^{-1} \otimes \mathbf{K}_3^{-1} \tag{11}$$

which reduces the complexity from $\mathcal{O}((NIJ)^3)$ to $\mathcal{O}(N^3 + I^3 + J^3)$. With this trick, we can conduct the computation in (8) efficiently. Denote eigendecomposition $\mathbf{K}_i = \mathbf{V}_i \mathbf{S}_i \mathbf{V}_i^\top$, we have

$$\Big(\mathbf{K} + \sigma^2 \mathbf{I}\Big)^{-1} = \mathbf{V}\Big(\mathbf{S} + \sigma^{-2} \mathbf{I}\Big)\mathbf{V}^\top \tag{12}$$

where $\mathbf{V} = \mathbf{V}_1 \otimes \mathbf{V}_2 \otimes \mathbf{V}_3$ and $\mathbf{S} = \mathbf{S}_1 \otimes \mathbf{S}_2 \otimes \mathbf{S}_3$. Similarly, we compute the determinant easily by

$$\log\det\Big(\mathbf{K} + \sigma^2 \mathbf{I}\Big) = \log\det\Big(\mathbf{S} + \sigma^2 \mathbf{I}\Big) = \sum_{k=1}^{NIJ} \log\Big(S_{kk} + \sigma^2\Big) \tag{13}$$

where S_{kk} is the kth diagonal element of $\mathbf{S}$. With the previous eigendecomposition done, the computation of the determinant only takes $\mathcal{O}(NIJ)$ time complexity. For implementation, the actual computation of $(\mathbf{K} + \sigma^2 \mathbf{I})^{-1}$ can take up a large space despite the fact that the time complexity is now reduced. Zhe et al. [16] discussed an efficient approach to complete the computation with space complexity of $\mathcal{O}(I^2 + J^2 + N^2)$, which is also implemented in this work.

We can now optimize the joint likelihood (8) with respect to hyperparameters in the kernel functions and the latent features $\mathbf{C}, \mathbf{P}, \mathbf{Z}$ using any gradient descent algorithm.

D. Model Inference

Given a new circuit design $\mathbf{z}_*$, the predictive distribution of any element in STA-matrix $\mathbf{Y}(\mathbf{z}_*)$ is a conditional Gaussian distribution [15]. For instance, for the ith row and the jth column of $\mathbf{Y}(\mathbf{z}_*)$, the predictive distribution is

$$\begin{aligned} y_{ij}(\mathbf{z}_*) &\sim \mathcal{N}\big(\mu_{ij}(\mathbf{z}_*), v_{ij}(\mathbf{z}_*)\big) \\ \mu_{ij}(\mathbf{z}_*) &= \mathbf{k}^\top(\mathbf{z}_*, i, j)\Big(\mathbf{K} + \sigma^2\Big)^{-1} \mathbf{y} \\ v_{ij}(\mathbf{z}_*) &= \hat{k}(\mathbf{z}_*, i, j) - \mathbf{k}^\top(\mathbf{z}_*, i, j)\Big(\mathbf{K} + \sigma^2\Big)^{-1} \mathbf{k}(\mathbf{z}_*, i, j) \end{aligned} \tag{14}$$

where $\hat{k}(\mathbf{z}_*, i, j) = k_1(\mathbf{z}_*, \mathbf{z}_*) k_2(\mathbf{c}_i, \mathbf{c}_i) k_3(\mathbf{p}_j, \mathbf{p}_j)$ and

$$\begin{aligned} \mathbf{k}(\mathbf{z}_*, i, j) &= \mathbf{k}_1^* \otimes \mathbf{k}_2^i \otimes \mathbf{k}_3^j \\ \mathbf{k}_1^* &= (k_1(\mathbf{z}_*, \mathbf{z}_1), \ldots, (\mathbf{z}_*, \mathbf{z}_N))^\top \\ \mathbf{k}_2^i &= (k_2(\mathbf{c}_i, \mathbf{c}_1), \ldots, (\mathbf{c}_i, \mathbf{c}_I))^\top \\ \mathbf{k}_3^j &= \big(k_3(\mathbf{p}_j, \mathbf{p}_1), \ldots, (\mathbf{p}_j, \mathbf{p}_J)\big)^\top. \end{aligned} \tag{15}$$

In practice, we follow [16] and use the Tucker operator to efficiently compute the predictive mean $\boldsymbol{\mu}^*$ with a time complexity of $\mathcal{O}(Nd(\sum_{k=1}^{Q} d_k + N))$.

Despite that the proposed method is flexible enough to use representative features $\mathbf{z}_*$ to make an accurate prediction of the STA-matrix $\mathbf{Y}(\mathbf{z}_*)$, it is unclear how such features $\mathbf{z}_*$ can be achieved for an unseen circuit design. One way to get them is probably using a graph network [18] with a significantly large dataset to learn a mapping from the netlist to $\mathbf{z}_*$, which is a promising method but is also a very challenging task itself due to the very high-computing demand for a very large graph. Instead, we provide an alternative way to leverage the local

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

information and infer $\mathbf{z}_*$ when some parts of matrix $\mathbf{Y}(\mathbf{z}^*)$ are observed. The inferred $\mathbf{z}_*$ is then substituted into (14) for the prediction of unseen parts of $\mathbf{Y}(\mathbf{z}^*)$ using the forward model.

However, the inference of $\mathbf{z}_*$ is not trivial and it does not admit a closed-form solution. Let us denote the observed part of $\mathbf{Y}(\mathbf{z}^*)$ as $\check{\mathbf{Y}}$ (of which the column is $\mathbf{x}$ in (1)), and the unobserved part as $\hat{\mathbf{Y}}$. To deliver the timing estimation, we can minimize the following objective function:

$$\arg\min_{\mathbf{z}_*} \|\mathbf{Y}(\mathbf{z}_*) - \check{\mathbf{Y}}\|_F^2 + \lambda \|\mathbf{z}_*\|_F^2 \tag{16}$$

where $\mathbf{Y}(\mathbf{z}_*)$ is the prediction based on (14) and λ is a hyperparameter to balance the contribution of the regularization term $\|\mathbf{z}_*\|_F^2$. When the optimal $\mathbf{z}_*$ is obtained, we can use (14) to predict $\hat{\mathbf{Y}}$.

Such a point estimation might not be robust to the noise in $\check{\mathbf{Y}}$. we can use a Laplace approximation [19] or variational approach to approximate the distribution given the observation $p(\mathbf{z}^*|\check{\mathbf{Y}})$ then compute the predictions by integrating out the estimated distribution

$$p\left(\hat{\mathbf{Y}}\right) = \int \hat{\mathbf{Y}}(\mathbf{z}^*) p\left(\mathbf{z}^*|\check{\mathbf{Y}}\right) d\mathbf{z}^*. \tag{17}$$

This integration is usually intractable and can be approximated by Monte Carlo sampling, which is a computationally expensive task despite of its elegance. For an efficient alternative solution to approximate the integral, we follow [20] to make predictions using a GP for $\hat{\mathbf{Y}}$ with $\check{\mathbf{Y}}$ as inputs. Such a GP model can be trained quickly with existing data $\mathcal{D}$ using the similar approach method in (1). Despite its simplicity, this approach works very well in our experiments.

E. Confidence Evaluation and Active Correction

There are many potential benefits associated with a predictive distribution (14). Here, we discuss some applications of using the posterior actually to enhance STA in practical usage with confidence evaluation. Given the STA prediction $p(y(\mathbf{z}_*, i, j)) \sim \mathcal{N}(\mu_{ij}(\mathbf{z}_*), v_{ij}(\mathbf{z}_*))$, we can compute the possibility of the prediction passing a legal check

$$l(\mathbf{z}_*, i, j) = p\left(y(\mathbf{z}_*, i, j) \geq y^0\right) = \Phi\left(\frac{\mu_{ij}(\mathbf{z}) - y^0}{v_{ij}(\mathbf{z})}\right) \tag{18}$$

where $\Phi(\cdot)$ is the cumulative density function (CDF) of a normal distribution and y^0 indicates the design margin tolerance that is set for a successful design in a specific technology node [11]. Based on $l(\mathbf{z}_*, i, j)$, we can further derive the possibility of all paths under the ith corner passing all STA checks by $o(\mathbf{z}_*, i) = \prod_{j=1}^{J} l(\mathbf{z}_*, i, j)$. With this indicator, the designer may choose to directly go back to the previous physical design phase for optimization when $o(\mathbf{z}_*, i)$ is very low rather than waiting for the completion of full-corner simulations that are almost certain to fail.

In the practical STA procedure, accuracy is crucially important. An ML pipeline can cause a catastrophe if it gives wrong predictions that guide the designer to make wrong modifications to the circuit. To prevent this issue, we can compute "prediction confidence" by computing the entropy [21] for each prediction

$$\begin{aligned} H(\mathbf{z}_*, i, j) = &-l(\mathbf{z}_*, i, j) \log(l(\mathbf{z}_*, i, j)) \\ &- (1 - l(\mathbf{z}_*, i, j)) \log(1 - l(\mathbf{z}_*, i, j)) \end{aligned} \tag{19}$$

which is a small value if the predictive confidence is high (i.e., the predictive variance is relatively small). For paths with high uncertainty (larger than a specified threshold H^0), we can actively double-check their predictions by executing the ground-truth STA tool for them to ensure the machine learning-based predictions do not generate misleading results.

F. Multiobjective Feature Engineering

To further enhance the performance of the timing prediction method, an effective way is to introduce automatic feature engineering, which generates high-dimensional features $\hat{\mathbf{x}}$ to replace the original features $\mathbf{x}$ as model inputs and improve the predictive accuracy with a significant margin. This approach remains effective even when we construct the mapping. using kernel methods as in GP. Despite that there are many existing automatic feature engineering approaches, e.g., Deep feature synthesis [22], Tsfresh [23], and AutoFeat [24], none of the state-of-the-art (SOTA) methods generalize well to our problem, which requires predictions under several corners. Our task is essentially a multiobjective automatic feature engineering problem, which is not well resolved in the automatic feature engineering literature. The task is nontrivial because the multiple objective data are correlated but different. This makes it difficult to measure the importance of a feature (which is an essential component of feature engineering), i.e., a feature can improve accuracy under corner A but degrade accuracy under corner B.

To solve this challenge, we propose multiobjective feature engineering (MoFe) to enable general automatic feature engineering for our problem. Denote the target (training) output $\mathbf{Y} \in \mathbb{R}^{N \times D}$ corresponding to inputs $\mathbf{X}$ here. Our method first finds its linear representations, which are ideally independent, i.e., solving

$$\arg\min_{\mathbf{V}} ||\mathbf{Y} - \mathbf{U}\mathbf{V}^\top||^2 \tag{20}$$

where $\mathbf{U} \in \mathbb{R}^{N \times R}$ is the coefficient matrix and $\mathbf{V} \in \mathbb{R}^{D \times R}$ are orthogonal bases (i.e., $\mathbf{V}^\top\mathbf{V} = \mathbf{I}$) that linearly project the coefficients onto the original space. In practice, this optimization is efficiently solved using singular value decomposition (SVD) as in principal component analysis (PCA) [25]

$$\mathbf{V}\mathbf{S}\mathbf{V}^\top = \left(\mathbf{Y} - \mathbf{1}\bar{\mathbf{y}}^\top\right)^\top \left(\mathbf{Y} - \mathbf{1}\bar{\mathbf{y}}^\top\right) \tag{21}$$

where $\bar{\mathbf{y}} = (1/N)\sum_{n=1}^{N} \mathbf{y}_n$ is the average vector of $\mathbf{Y}$; $\mathbf{1}$ is a vector with all elements being 1; $\mathbf{V}$ are the eigenvector collection and $\mathbf{S}$ are the corresponding eigenvalues. The eigenvalues are sorted in descending order such that the variance of $\mathbf{Y}$ is dominantly explained by the first few principal directions. The coefficients are computed by $\mathbf{U} = \mathbf{Y}\mathbf{V}$ since $\mathbf{V}$ is orthonormal. Instead of applying automatic feature engineering to $\mathbf{Y}$ directly, we work on the coefficients $\mathbf{U}$. Because the principal directions are orthogonal, the coefficients are

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETERS IN ALGORITHMS 1 AND 2

u_r	The r-th column of U, , representing the r-th new feature
D	The number of columns in Y
J	The base-2 logarithm of the total number of data block 2^J
s_r	The r-th column of S and the corresponding eigenvalue of u_r

Algorithm 1 FeatureRank

Require: Dateset: $\{\mathbf{X}, \mathbf{u}_r\}$, candidate feature $\mathbf{X}'$

Train LightGBM $\mathcal{M}_0$ on $\{\mathbf{X}, \mathbf{u}_r\}$ with 5-fold cross-validation and generate loss l_0
for $d = 1$ to D **do**
  Continue training LightGBM $\mathcal{M}_0$ with new features $\{[\mathbf{X}; \mathbf{x}'_{:d}], \mathbf{u}_r\}$ and generate loss l_d
end for
return Feature Importance $\mathbf{e}_r = l_0/\mathbf{l}$ ($\mathbf{l} = [l_1, \cdots, l_D]^\top$)

Algorithm 2 Multiobjective Feature Engineering

Require: Training data $\mathbf{X}$, $\mathbf{Y}$, # data block J

Use PCA (Eq. (21)) to obtain coefficients $\mathbf{U}$ and variance vector $\mathbf{s}$
for $r = 1, \ldots, D$ **do**
  Initialize base feature $\mathbf{X}_r \leftarrow \mathbf{X}$
  for $k = 1, \ldots, K$ **do**
    Generate candidate features $\mathbf{X}'_r \leftarrow \boldsymbol{\phi}(\mathbf{X}_r)$
    for $j = 1, \ldots, J$ **do**
      $\mathbf{e}_r = \text{FeatureRank}(\{\mathbf{X}, \mathbf{u}_r, \mathbf{X}'_r\}_{1:2^j})$;
      Keep top-ranked features $\mathbf{X}'_r \leftarrow \text{TopHalf}(\mathbf{X}'_r, \mathbf{e}_r)$
    end for
    Update base feature $\mathbf{X}_r \leftarrow [\mathbf{X}_r; \mathbf{X}'_r]$;
  end for
  $\mathbf{e}_r = \text{FeatureRank}(\{\mathbf{X}, \mathbf{u}_r, \mathbf{X}'_r\})$;
  Scale feature importance $\mathbf{e}_r \leftarrow \mathbf{e}_r \cdot s_r$
end for
return Features and importance $\{\mathbf{X}'_r, \mathbf{e}_r\}_{r=1}^{D}$

statistically independent of each other. Thus, we can apply automatic feature engineering to each coefficient individually. The key ingredient of MoFe is to apply automatic feature engineering to each coefficient individually first and compare their importance later based on the feature significance and the principal direction variance.

We now discuss the feature engineering part in detail inspired by OpenFE [26], which use an expand-reduce framework to first generate (expand) as many possible candidate features and remove the less relevant candidate features. First we design a feature engineering function ϕ that enumerates all first-order transformations of the basic features using given operators to create a huge candidate feature pool. Appendix contains the list of operators and how we use them to transform numerical features. For instance, if the basic features are $x^{(1)}$ and $x^{(2)}$, the candidate feature pool becomes $\{x^{(1)} + x^{(2)}, x^{(1)} - x^{(2)}, x^{(1)} * x^{(2)}, x^{(1)}/x^{(2)}, x^{(2)}/x^{(1)}, (x^{(1)})^2, (x^{(2)})^2\}$ given the operator set $\{+, -, *, /, x^2\}$.

Generating the feature pool is fast and straightforward. The challenge is to keep only the most important features that generate the best-predictive accuracy. To this end, we use a gradient-boosting-like method to test how much a given feature can improve the predictive accuracy and determine its importance. More specifically, we first train a LightGBM $\mathcal{M}_0$ on data $\{\mathbf{X}, \mathbf{u}_r\}$ using fivefold cross-validation, where $\mathbf{u}_r$ is the rth column of $\mathbf{U}$. Based on $\mathcal{M}_0$, we test all features in the candidate feature pool individually and concludes their importance based on the accuracy improvement (computed on the validation set). Since the model is trained on $\mathcal{M}_0$ with only a new feature each time, this approach is fast and can be applied to a large number of candidate features. We call this Feature Ranking, which is a key component of MoFe and is detailed in Algorithm 1. The specific meanings of the parameters can be found in Table I.

Although feature ranking provides an effective method for estimating the incremental performance of new features, it is still computationally expensive to deal with a large number of candidate features. In order to improve the practicality, we introduce a successive halving approach. We divide the total samples into 2^J blocks, each with training and validation set at a ratio of 9:1. We incrementally expand the data size from one block to all blocks, which is used to rank the candidate features and remove the ineffective ones by half in each iteration. This stage aims at rough screening, where we assess the incremental effect of individual new features. Therefore, our method runs for all blocks until 2^j.

The limitation with the above approach is that the candidate features are based on the existing features; the higher-order features (features generated based on generated features) are not considered. To address this issue, we repeat the previous process K times, and each time we use the features generated in the previous iteration as the existing features to generate new features.

The process is repeated for each column of $\mathbf{U}$ to get features and their importance scores $\{\mathbf{X}'_r, \mathbf{e}_r\}$. The importance scores to the multioutput $\mathbf{Y}$ is then calculated by $\mathbf{e}_r \cdot s_r$, where s_r is the rth column of S and the corresponding eigenvalue of u_r. This is the complete process of the MoFe algorithm which is summarized in Algorithm 2. Base on the result, we can select the top n features with the highest-importance scores and add them to the input to our model. Although our model only used LightGBM in the feature selection stage and did not use it for actual prediction, we will show in the experiment that the features selected by MoFe can significantly improve the performance of the model.

IV. EXPERIMENT RESULTS

A. Experimental Setup

We evaluate the performance of the proposed technique against the machine learning models used in the previous work [10], [11] including ridge regression (RR), multilayer perceptron (MLP), and random forest (RF). The original multivariate linear regression in [10] is not used because it is a vanilla version of RR and may suffer from overfitting.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PARAMETERS OF FREEPDK 45-NM TECHNOLOGY PROCESS CORNERS

	Corner1	Corner2	Corner3	Corner4	Corner5	Corner6	Corner7
Temperature (°C)	25	0	125	-40	-40	-40	-40
Supply voltage (V)	1.1	1.25	0.95	1.25	1.25	1.25	0.95
V_{thp} (V)	-0.5044	-0.4794	-0.5294	-0.5044	-0.4794	-0.5294	-0.5044
V_{thn} (V)	0.6078	0.5828	0.6328	0.6078	0.5828	0.6328	0.6078
	Corner8	Corner9	Corner10	Corner11	Corner12	Corner13	Corner14
Temperature (°C)	-40	-40	0	0	125	25	25
Supply voltage (V)	0.95	0.95	1.25	1.25	0.95	1.1	1.1
V_{thp} (V)	-0.4794	-0.5294	-0.5294	-0.5044	-0.4794	-0.4794	-0.5294
V_{thn} (V)	0.5828	0.6328	0.6328	0.6078	0.5828	0.5828	0.6328

TABLE III
IWLS BENCHMARKS USED IN OUR EXPERIMENTS

	sequential	inverter	buffer	logic	total
itc99_b17	1,415	4,564	319	30,819	37,117
itc99_b18	3,296	1,1429	407	7,6916	92,048
itc99_b19	6,594	20,866	439	146,258	174,157

We thus adopt the more sophisticated RR as an alternative as a representation of the linear regression-based method. Our method is implemented based on modern machine learning library PyTorch [27] whereas the other baseline methods are implemented on scikit library [28] as suggested in [11].

To prepare the evaluation data, we use the publicly available FreePDK 45-nm technology and Nangate 45-nm standard cell library to generate design data for model training and validation. The process corners included in the standard cell library are used in our following experiments. In order to mimic the multicorner cases in real deep submicron design environment, we generated 14 process corners based on the limited process corners provided in the PDK. The process corner parameters of 45-nm technology nodes are listed in Table II. With these parameters, we used Cadence Elaborate for library characterizations and generate the timing libraries of 14 process corners.

We follow [11] to use the IWLS benchmarks. From these benchmarks, we choose b17, b18 and b19 (see Table III for some details) from the ITC99 benchmark suites. These benchmarks are first synthesized by Cadence RTL compiler and then analyzed by OpenTimer [29] to generate the STA timing results for model training and validation.

For each circuit, we first obtain the timing critical path delays and set a strict timing constraint appropriately such that the timing approaches closure after several iterative timing optimizations. During the iterative timing optimizations, several ECO commands (such as buffer insertion and path splitting) are executed to change the circuit structure to improve timing performance. Each iteration generates a new circuit version and the timing critical path delays under selected process corners (no. 1–5) are used for model training. To stay consistent with the previous work [11], we use mean absolute error (MAE) and LESS10, which represents the prediction error is less than 10% times of the true value.

In each iteration, we require a certain amount of time to complete the timing analysis under PCs. Since the required time in each iteration does not vary significantly, we take the average of the STA analysis time required in five physical design stages of b17 as the "run time of prediction" for generating timing-critical path slack using traditional STA tools, as shown in Table IV. We use the average of the times predicted by our model as the STA prediction run time of our method. Here, "1 corner/3 corners/5 corners" represents using timing analysis results from 1/3/5 known corners to predict timings for the remaining unknown process corners. The term "Training time" indicates the time required by different models during the training process. Similarly, we calculate the average prediction times for RL, MLP, and RF. Although the proposed method takes slightly longer time, our accuracy is far higher than other methods, and it can also provide confidence evaluations.

B. Timing Prediction Accuracy Comparisons

Our proposed technique is first compared with RR, MLP, and RF in terms of prediction accuracy. We conduct the experiment task with the case studies: 45-nm STA results with a training-testing ratio of 0.7–0.3 using different paths from the same circuit. The detailed results of MAE and LESS10 are shown in Fig. 3. We can observe that the proposed technique outperforms the competitors in almost all cases significantly. Compared to RR, MLP, and RF, our prediction accuracy can improve by 17.1%, 23.9% and 29.2% in terms of LESS10, respectively. Notably, with more corner data available for training, the prediction accuracy improvement of our method is clearly more significant, indicating a stronger model capability to harness the correlations in data. The average results for LESS10 and MAE are shown in Fig. 4. We can observe that as the number of training corners increases from 1 to 5, LESS10's prediction accuracy increases by 34.2% on average, indicating that more results from different corners acting as input features can help the model to learn the timing results with higher accuracy. Noticeably, we can also see that more corners do not always lead to better-prediction accuracy. For example,

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

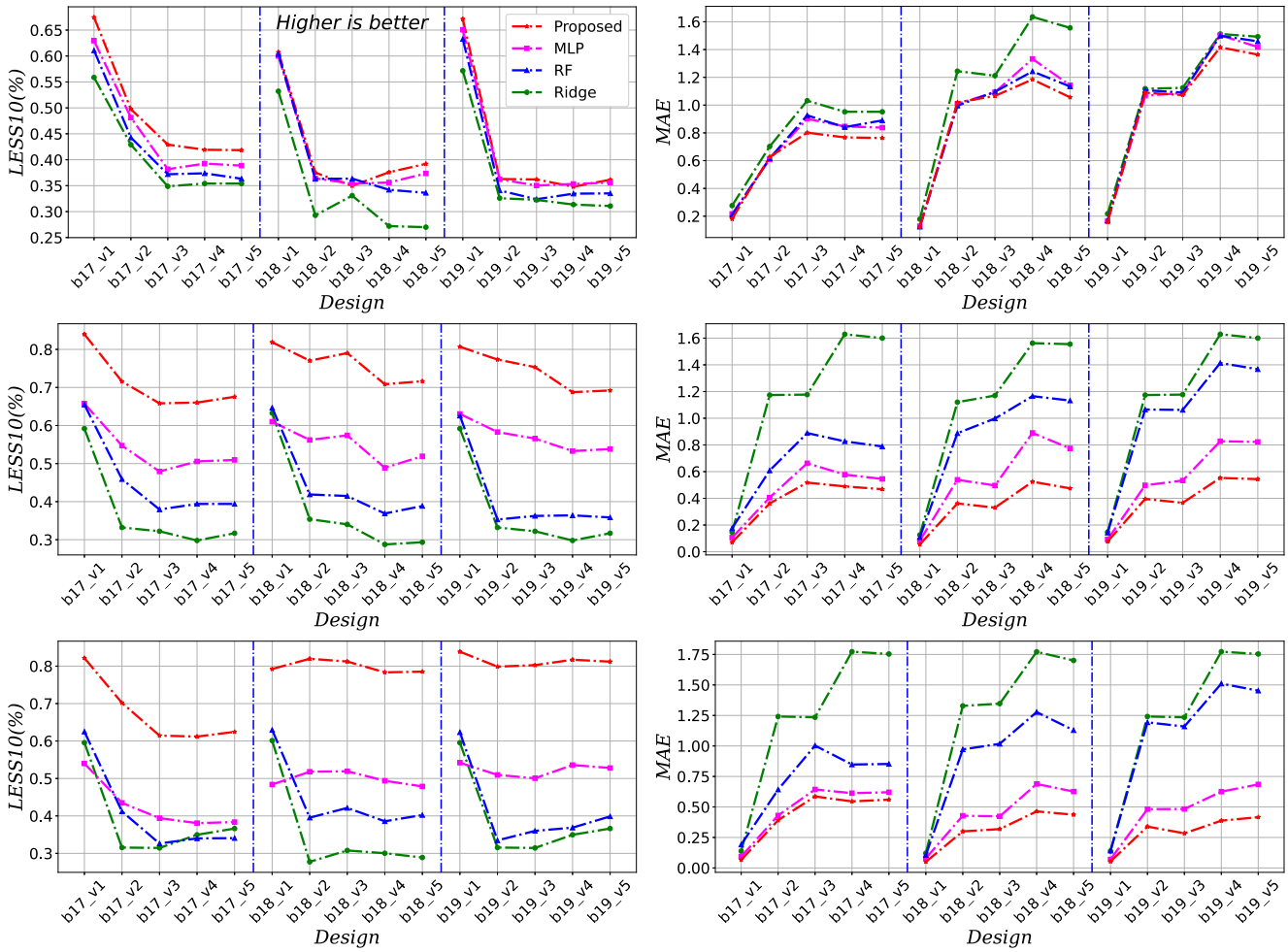

Fig. 3. LESS (left) and MAE (right) with $\{1, 3, 5\}$ dominant corners (from top to bottom) for different optimization versions of three benchmark circuits with 45-nm technology.

TABLE IV
TIMING PREDICTION RUN TIME AND MODEL TRAINING TIME COMPARISONS OF B17

	Timing Prediction Run Time (s)			Training Time (s)		
STA Analysis by vendor tool	3024.29961			N/A		
	1 corner	3 corners	5 corners	1 corner	3 corners	5 corners
MLP	0.01597	0.01168	0.00779	16.02157	14.77355	12.11831
Ridge	0.00099	0.00044	0.00099	0.00661	0.01056	0.02014
RF	0.00689	0.00617	0.00597	0.09216	0.09687	0.10120
Our Method	0.11000	0.08999	0.04158	26.32347	22.04874	15.93916

in the case of b17, the prediction accuracy of LESS10 decreases when the number of corners increases from 3 to 5, possibly because the training samples are now insufficient to train a more complex model without overfitting. This also highlights the importance of a feature engineering technique and a flexible model to make the best use of the available data.

We also tabulate the mean and max MAE improvements of our proposed technique compared to the baseline methods in Table V. We can observe that our proposed technique demonstrates a significant MAE improvement over the SOTA methods with a maximum of 4.57x. We can also observe that more corners and more training data will boost our proposed technique for a more significant improvement, implying the effectiveness of the model.

C. STA Prediction With Timing Optimization Iterations

In practical timing optimizations, designers normally iteratively alter a design to approach timing closure. This process generates similar STA-matrices, which can be used to enhance the proposed technique's capability by knowledge transferred from previous timing optimization iterations. Furthermore, our model has the prediction capability with

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

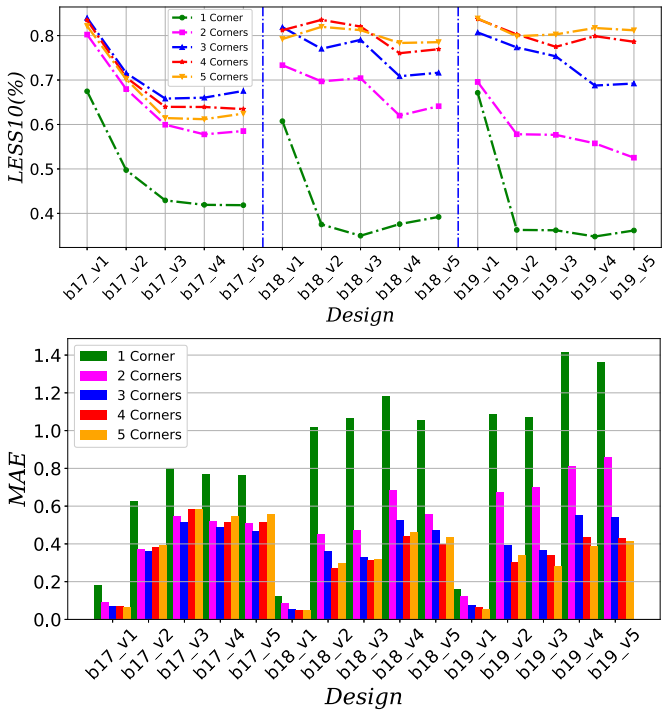

Fig. 4. Average LESS10/MAE of $\{1, 2, 3, 4, 5\}$ corners where the training-testing ratio is $0.7 - 0.3$ in b17, b18, and b19.

TABLE V
MAE IMPROVEMENTS OF THE PROPOSED METHOD OVER THE SOTA METHODS [10], [11]

Case		1 corner	3 corners	5 corners
Ridge	Mean	1.19x	3.04x	3.57x
	Max	1.53	3.54x	4.57x
MLP	Mean	1.05x	1.41x	1.34x
	Max	1.20x	1.39x	1.70x
RF	Mean	1.05x	2.26x	2.60x
	Max	1.17x	3.02x	4.08x

only timing results of a part of observed critical timing paths to predict the rest of the critical path timing. More specifically, we use $[\mathbf{y}(\mathbf{z}_*, i, 1), \ldots, \mathbf{y}(\mathbf{z}_*, i, R)]$ to predict $[\mathbf{y}(\mathbf{z}_*, i, R+1), \ldots, \mathbf{y}(\mathbf{z}_*, i, J)]$ with timing results from previous optimization iterations as training data. This is equivalent to conducting the classic STA prediction [11] with the transposed STA-matrix.

To evaluate such a situation, we conducted our experiments on b17, b18, b19 with five timing optimization iterations with 45-nm technology node. We now try to predict the timing of the fifth iteration based on previous timing results. At each iteration, the timing results of all previous iterations are used as training data to improve the model's accuracy. We conduct the experiments on b17, b18, b19 with 45-nm technology node and the detailed results of LESS10 are shown in Fig. 5, where the LESS10 with more iterations as training data is shown. We can see that our technique can effectively use previous knowledge to deliver accurate predictions. In particular, we can see that the prediction accuracy gradually increases with the number of iterations, indicating that our model can effectively utilized the results from the timing results. This is particularly useful in the timing optimization procedure, where the optimization process can now be accelerated by using the timing results of previous iterations to predict the timing results of the next iteration. It is also noticeable that the prediction accuracy of b19 suddenly drops at the 3rd iteration, which is possibly because the circuit undergoes a significant change in the 3rd iteration, which makes the previous timing results less useful. Nevertheless, the performance quickly improves in the next iteration, which shows the robustness of our model.

TABLE VI
LESS10 AND MAE IN THE ABSENCE OF A TRAINING SET FOR PREDICTING MATRICES

	Proposed	MLP	RF	Ridge
LESS10 (%)	70.46	58.61	44.22	48.09
MAE	0.4224	0.5095	0.7022	0.7191

Furthermore, if we do not utilize partial information from the target timing results, while using another previous fully known timing matrices as the training set, we would still obtain a similar prediction accuracy, with an average LESS10 of 70.46% in the case of b17 with three dominant corners. As shown in Table VI, in this scenario, our method's accuracy is still higher than other methods. However, by incorporating partial known information from the target timing results, along with previous timing results, the prediction accuracy can be significantly improved.

D. Reliable Evaluations With Active Correction

To further evaluate the reliability of our proposed technique, we first examine the reliability of the predictive posterior distribution of our model by comparing the predicted confidence interval with the true value. Particularly, we will compare the truth value with the predicted mean and variance of the Gaussian distribution to see if the truth value is within the 95% confidence interval of the predicted value. If that is the case, it means that the predicted posterior distribution is reliable. We perform this evaluation and denote the percentage of paths that are within the 95% confidence interval of the predicted values, which we call this the proportion within 2σ accuracy. The results for b17, b18, b19 with 45-nm technology node with 1 corner, 3 corners, and 5 corners are shown in Fig. 6, where the proportion within 2σ accuracy is shown. We can see that the proportion within 2σ accuracy is about 94.3% for all cases even with only 1 corner, which indicates that our model can reliably predict the timing of a critical path. As the number of corners increases, the proportion within 2σ accuracy also increases, indicating that the increase of the number of corners helps to improve the accuracy and reliability of the prediction. Averagely, with five corners, the proportion within 2σ accuracy is about 97.1%, which is quite significant considering the fact that the timing of a critical path is a very sensitive value.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

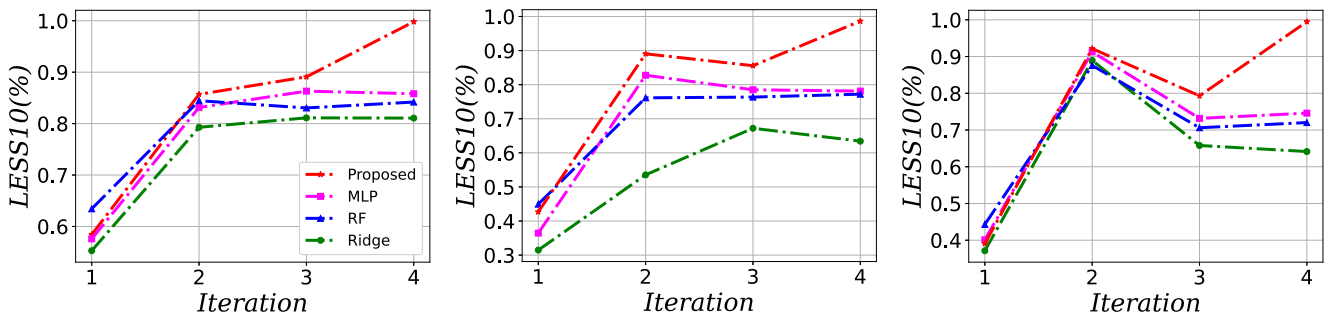

Fig. 5. LESS10 of b17, b18, and b19 with previous timing iteration information.

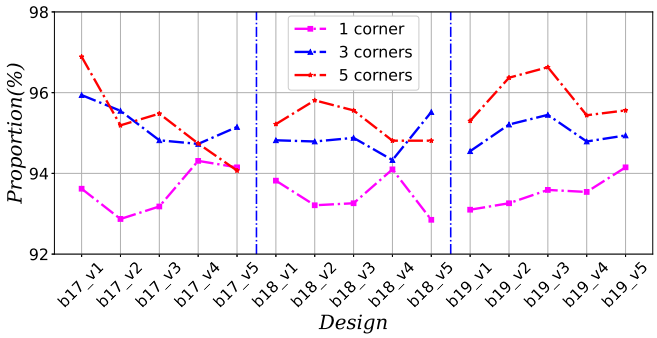

Fig. 6. Proportion of Confidence Intervals of $\{1, 3, 5\}$ corners for all benchmarks with 45-nm technology node.

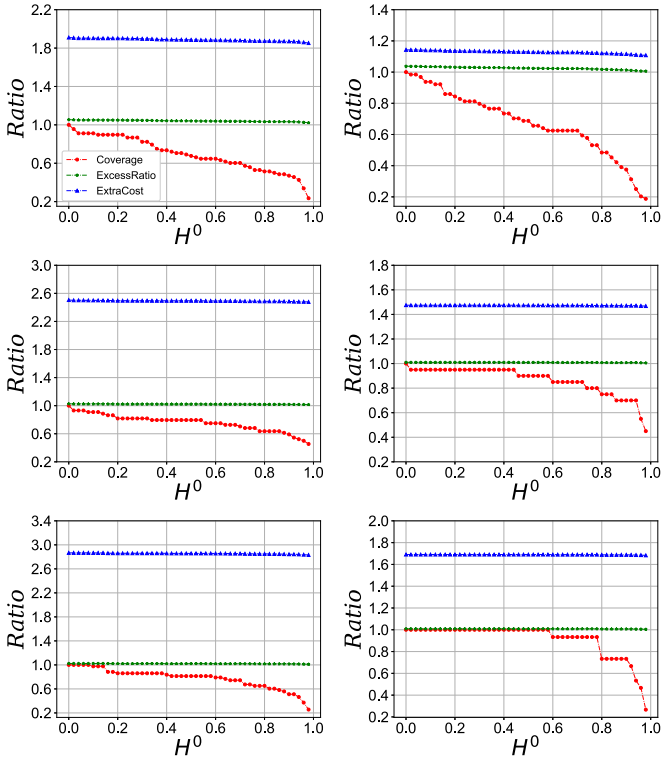

Fig. 7. Key indicators of $\{3, 5\}$ corners (from left to right) in b17, b18, and b19 (from top to bottom).

However, the 97.1% proportion within 2σ accuracy is probably not enough to guarantee the reliability in practical applications. To further demonstrate the advantage of having a predictive posterior as discussed in Section III-E, we conduct the confidence evaluation with the active correction process discussed in Section III-E to run STA for the paths with high possibility of being crucially wrong, i.e., those paths with a large chance to fail the STA. This process will increase the computational cost but guarantee the reliability of the prediction. We define three key indicators to examine the performance: 1) Coverage, the ratio of the wrong predictions correctly marked divided by the total number of wrong predictions (higher is better); 2) ExcessRatio, the total number of correctly and incorrectly marked paths divided by the number of wrong predictions (lower is better); and 3) ExtraCost, the total number of marked paths, which need rerun of STA for these points, divided by the initial STA runs (lower is better).

We conduct this active correction process on the b17, b18, b19 benchmarks with 45-nm technology node and three corners and five corners. The results are shown in Fig. 7, where the coverage, excess ratio, and extra cost are shown. As expected, a smaller threshold H^0 can improve coverage to near 1 (indicating a 100% reliable correction for all wrong predictions), but it will also increase the ExcessRatio and ExtraCost. Nevertheless, we can find that the ExtraCost grows much slower than the Coverage, which means that we can achieve a high coverage with a small ExtraCost. In practice, we will suggest a small threshold H^0 to achieve a high coverage to ensure the reliability with a certain ExtraCost. As the number of corners increases, the extra cost decreases significantly. This demonstrates that with an increasing number of known corners, we can achieve more reliable corrections with fewer costs for all cases.

E. Ablation Study of MoFe

As stated in Section III-F, feature engineering is used to improve our model despite that our method is not based on LightGBM. In this section, we assess the effectiveness of feature engineering by comparing the prediction accuracy of our method with and without feature engineering for experiments in Section IV-B. The results of LESS10 are shown in Fig. 8. In the figures for different experiment setups, we can clearly see that the LESS10 training result with feature engineering is consistently better than that without feature engineering. There is no performance degradation in the case

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

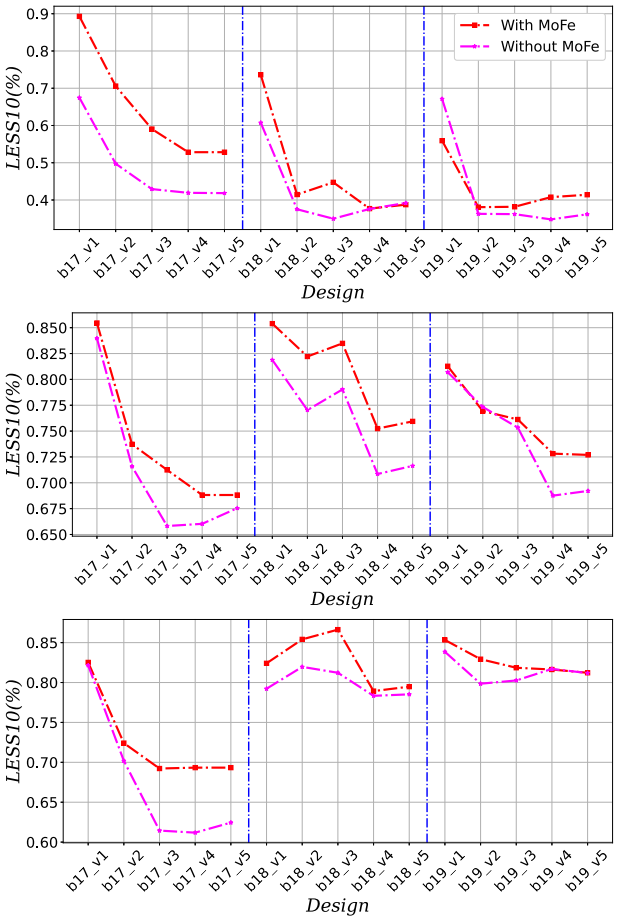

Fig. 8. Average LESS10 of $\{1, 3, 5\}$ corners in b17, b18, and b19 with and without feature engineering.

of using feature engineering, and the average improvement is 4.7% over all design cases. Our work can be replicated by using the same dataset and training methodology, which will result in obtaining the same features automatically. To better showcase the features we generated, we list the features generated in the timing prediction of b17. We have specified the selection of the top 20 features: $(l_2 + l_3)$, $\text{abs}(l_3)$, (l_1/l_2), $\text{residual}(l_2)$, $\max(l_1, l_2)$, (l_1/l_3), $\min(l_1, l_2)$, $(l_2 - l_3)$, $(l_1 - l_3)$, $(l_1 + l_2)$, $(l_1 + l_3)$, $\text{abs}(l_1)$, $(l_1 - l_2)$, $(l_1 * l_3)$, $(l_1 * l_2)$, $(l_2 * l_3)$, $\text{residual}(l_3)$, $\text{residual}(l_1)$, $\text{freq}(l_1)$, and $\min(l_1, l_3)$, where l_1 represents the first column, l_2 represents the second column, and l_3 represents the third column. The definitions of the operators are in the Appendix.

V. Conclusion

We propose a novel nonlinear machine learning-based timing prediction model. The model takes latent features of process corners, path timings and circuit information in different timing optimization iterations to improve the prediction accuracy. Then, the model is equipped with an active correction and feature engineering to obtain higher-prediction accuracy. Experimental results demonstrate the model's superiority in terms of accuracy and scalability compared to the SOTA work. The future work includes improving the proposed technique to deal with iterative timing optimizations with a varying number of timing critical paths and incorporating circuit topology information to enhance our methodology in real circuit design with advanced technology node.

Appendix
Operators and Feature Transformations

We enumerate all the first-order transformations to form the candidate feature set. A first-order transformation uses one operator once to transform base features.

We list the details of all the operators below. f stands for a numerical feature.

1) $\text{Freq}(f)$: The frequency of feature f.
2) $\text{Abs}(f)$: Element-wise absolute value.
3) $\text{Log}(f)$: Element-wise logarithm.
4) $\text{Sqrt}(f)$: Element-wise square root.
5) $\text{Sigmoid}(f)$: Element-wisely apply function $x \mapsto (1/1 + e^{-x})$.
6) $\text{Round}(f)$: Element-wise rounding.
7) $\text{Residual}(f)$: Element-wisely take decimal part.
8) $\text{Min}(f_1; f_2)$: Element-wise minimum.
9) $\text{Max}(f_1; f_2)$: Element-wise maximum.
10) $f_1 + f_2$: Element-wise addition.
11) $f_1 - f_2$: Element-wise subtraction.
12) $f_1 \times f_2$: Element-wise multiplication.
13) $f_1 \div f_2$: Element-wise division.

References

[1] J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs: A Practical Approach*. New York, NY, USA: Springer, 2009.
[2] A. K. M. M. Islam and H. Onodera, "pvt^2: Process, voltage, temperature and time-dependent variability in scaled CMOS process," in *Proc. Int. Conf. Comput.-Aided Design*, 2018, pp. 1–6.
[3] J. Hu, D. Sinha, and I. Keller, "Tau 2014 contest on removing common path pessimism during timing analysis," in *Proc. Int. Symp. Phys. Design*, 2014, pp. 591–591.
[4] J. Hu, G. Schaeffer, and V. Garg, "Tau 2015 contest on incremental timing analysis: Incremental timing and CPPR analysis," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design*, 2015, pp. 882–889.
[5] D. Payne. "How 16-nm and 14-nm FinFETs require new SPICE simulators." 2016. [Online]. Available: https://semiwiki.com/finfet/5454-how-16nm-and-14nm-FinFETs-require-new-spice-simulators/.
[6] L. G. E. Silva, L. M. Silveira, and J. R. Phillips. "Efficient computation of the worst-delay corner," in *Proc. Design, Autom. Test Eur. Conf. Exhib.*, 2007, pp. 1–6.
[7] S. Onaissi and F. N. Najm, "A linear-time approach for static timing analysis covering all process corners," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 7, pp. 1291–1304, Jul. 2008.
[8] M. Hashimoto and H. Onodera, "A performance optimization method by gate sizing using statistical static timing analysis," in *Proc. Int. Symp. Phys. Design ISPD*, 2000, pp. 111–116.
[9] C. Ababei and K. Bazargan, "Statistical timing driven partitioning for VLSI circuits," in *Proc. Design, Autom. Test Eur. Conf. Exhib.*, 2000, p. 1109.
[10] A. B. Kahng, U. Mallappa, L. Saul, and S. Tong, ""Unobserved corner" prediction: Reducing timing analysis effort for faster design convergence in advanced-node design," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, 2019, pp. 168–173.
[11] Z. Zhao et al, "Machine-learning-based multi-corner timing prediction for faster timing closure," *Electronics*, vol. 11, no. 10, p. 1571, 2022.
[12] K-J. Chang, L.-F. Chang, R. G. Mathews, and M. G. Walker, "Method and system for extraction of parasitic interconnect impedance including inductance," U.S. Patent, 6 643 831, Nov. 4, 2003.
[13] W. Deng, K. Okada, and A. Matsuzawa, "A feedback class-C VCO with robust startup condition over PVT variations and enhanced oscillation swing," in *Proc. (ESSCIRC)*, 2011, pp. 499–502.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.

[14] W. Liu, J. C. Príncipe, and Simon Haykin, "Kernel least-mean-square algorithm," in *Kernel Adaptive Filtering: A Comprehensive Introduction*, Hoboken, NJ, USA: Wiley, 2010.

[15] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: MIT, 2006, p. 266.

[16] S. Zhe, W. Xing, and R. M. Kirby, "Scalable high-order Gaussian process regression," in *Proc. 22nd Int. Conf. Artif. Intell. Statist.*, 2019, pp. 2611–2620.

[17] M. G. Genton. "Classes of kernels for machine learning: A statistics perspective," *J. Mach. Learn. Res.*, vol. 2, no. 2, pp. 299–312, 2002.

[18] Z. Guo, M. Liu, J. Gu, S. Zhang, D. Z. Pan, and Y. Lin, "A timing engine inspired graph neural network model for pre-routing slack prediction," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, 2022, pp. 1207–1212.

[19] N. E. Breslow, and D. G. Clayton, "Approximate inference in generalized linear mixed models," *J. Amer. Statist. Assoc.*, vol. 88, no. 421, pp. 9–25, 2012.

[20] C. Henrik Ek, P. H. S. Torr, and N. D. Lawrence, "Gaussian process latent variable models for human pose estimation," in *Proc. Int. Workshop Mach. Learn. Multimodal Interact.*, Brno, Czech Republic, 2007, pp. 132–143.

[21] S. Goldstein and J. L. Lebowitz, "On the (Boltzmann) entropy of non-equilibrium systems," *Physica D Nonlin. Phenom.*, vol. 193, no. 1, pp. 53–66, 2004.

[22] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. IEEE Int. Conf. Data Sci. Adv. Anal. (DSAA)*, 2015, pp. 1–10.

[23] M. Christ, N. Braun, J. Neuffer, and A. W. Kempa-Liehr, "Time series feature extraction on basis of scalable hypothesis tests (tsfresh—a python package)," *Neurocomputing*, vol. 307, pp. 72–77, Sep. 2018.

[24] F. Horn, R. Pack, and M. Rieger, "The autofeat python library for automated feature engineering and selection," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discov. Databases*, 2019, pp. 111–120.

[25] K. Pearson, "Liii. On lines and planes of closest fit to systems of points in space," *London, Edinburgh, Dublin Philos. Mag. J. Sci.*, vol. 2, no. 11, pp. 559–572, 1901.

[26] T. Zhang, "OpenFE: Automated feature generation beyond expert-level performance," 2022, *arXiv:2211.12507*.

[27] A. Paszke, "Automatic differentiation in PyTorch," in *Proc. 31st Conf. Neural Inf. Proc. Syst.*, 2017, pp. 1–4.

[28] F. Pedregosa et al., "Scikit-learn: Machine learning in python," *J. Mach. Learn. Res.*, vol. 12, no. 85, pp. 2825–2830, 2012.

[29] T. W. Huang and M. D. F. Wong. "OpenTimer: A high-performance timing analysis tool," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2015, pp. 895–902.

Wei W. Xing (Member, IEEE) received the Ph.D. degree in engineering from the University of Warwick, Coventry, U.K., in 2017.

He was a Postdoctoral Fellow with the Scientific Computing Institute, The University of Utah, Salt Lake City, UT, USA, until 2020. He is the author of more than 45 peer-reviewed papers including DAC, ASP-DAC, NeurIPS, AAAI, *Journal of Cybersecurity and Privacy*, *Journal of Positive Sciences*, and EST. He is currently an Assistant Professor with Beihang University, Beijing, China. His research interest is mainly in reliable machine learning for EDA and scientific computing, AI4science, and digital twins.

Longze Wang (Student Member, IEEE) received the B.S. degree from Beihang University, Beijing, China, in 2013, where he is currently pursuing the M.S. degree in electrical engineering.

His research interests include application of machine learning in static timing analysis.

Zhelong Wang (Student Member, IEEE) received the B.S. degree in electronic science and technology engineering from the Wuhan University of Technology, Wuhan, China, in 2022, where he is currently pursuing the M.S. degree in electronic science and technology engineering.

His research focuses on static timing analysis and digital integrated circuit backend design.

Zhaoyu Shi (Student Member, IEEE) received the B.E. degree from the China University of Geosciences, Beijing, China, in 2021. He is currently pursuing the M.S. degree in electronic science and technology with Beihang University, Beijing.

His research interests include machine learning, integrated storage and computing architecture design, and feature engineering.

Ning Xu (Member, IEEE) received the Ph.D. degree in electronic science and technology from the University of Electronic Science and Technology of China, Chengdu, China, in 2003.

He was then a Postdoctoral Fellow with Tsinghua University, Beijing, China, from 2003 to 2005. He is currently a Professor with the School of Information Engineering, Wuhan University of Technology, Wuhan, China. His research interests include EDA, image processing, and artificial intelligence.

Yuanqing Cheng (Senior Member, IEEE) received the Ph.D. degree from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2012.

He is currently a Tenured Associate Professor with Beihang University, Beijing. He has published more than 50 peer review papers including *Proceedings of the IEEE*, DAC, ICCAD, IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION, and IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS. His research interests include reliable and physical design for 3-D integrated circuits, reliable, and low-power design for emerging technologies, such as spintronics and carbon nanotube technologies.

Weisheng Zhao (Fellow, IEEE) received the Ph.D. degree in physics from the University of Paris Sud, Paris, France, in 2007.

In 2009, he was a Tenured Research Scientist with French National Research Center. Since 2014, he has been a Distinguished Professor with Beihang University, Beijing, China, where he is currently a Professor with the School of Integrated Circuit Science and Engineering. He has published more than 200 scientific articles in leading journals and conferences, such as *Nature Electronics*, *Nature Communications*, *Advanced Materials*, IEEE TRANSACTIONS, ISCA, and DAC. His current research interests include the hybrid integration of nanodevices with CMOS circuit and new nonvolatile memory (40-nm technology node and below), such as MRAM circuit and architecture design.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:50:36 UTC from IEEE Xplore. Restrictions apply.